



Licenciatura em Telecomunicações e Informática

Arquitetura e Tecnologia de Computadores

1st Semester 2024/2025

7 seg display + PBs (GPIO)

Tiago Gomes

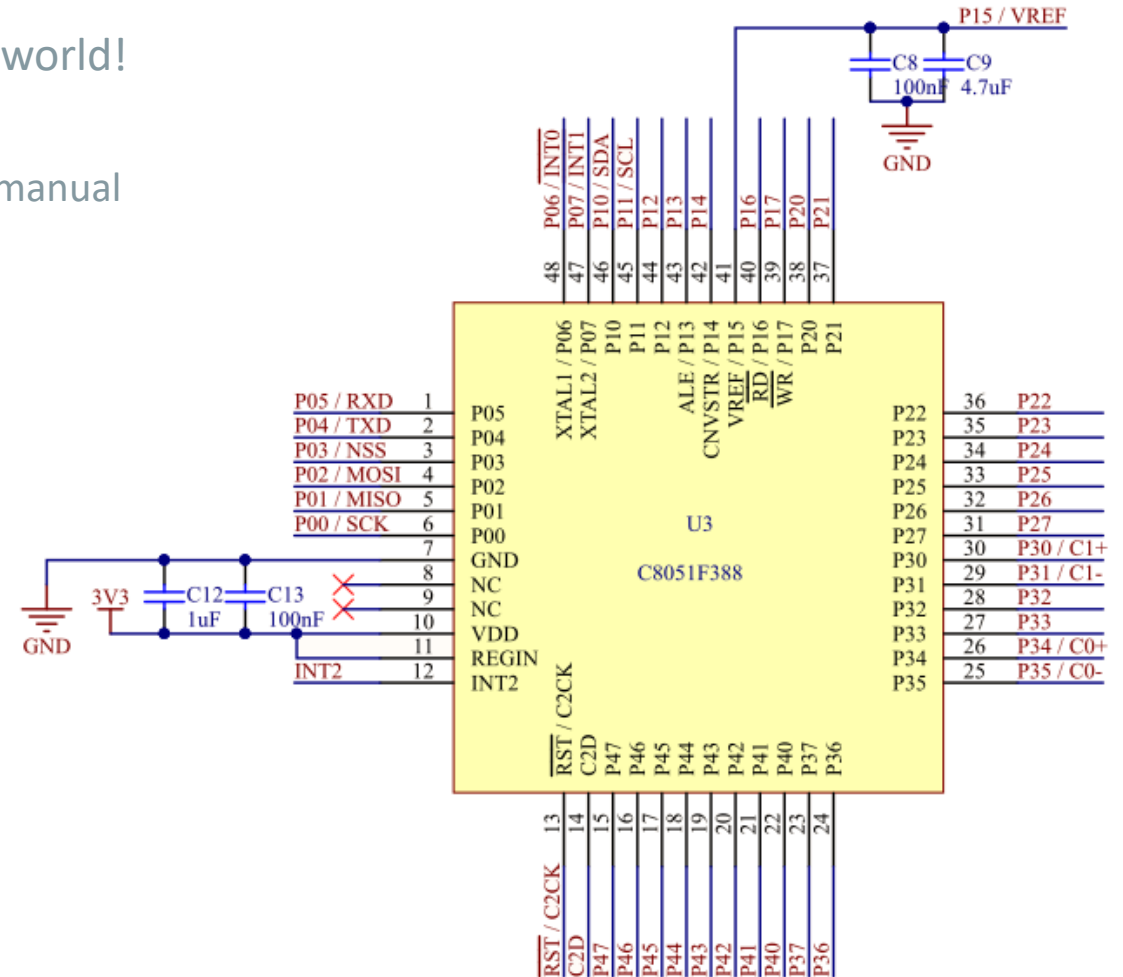
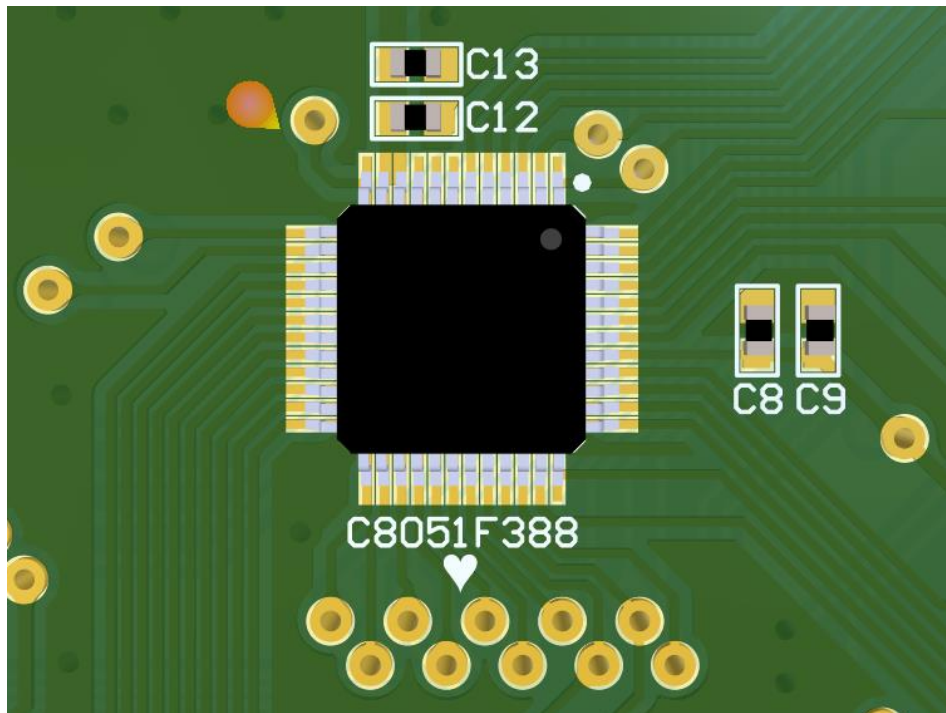
Embedded Systems Research Group (ESRG)
Centro ALGORITMI, Universidade do Minho

mr.gomes@dei.uminho.pt



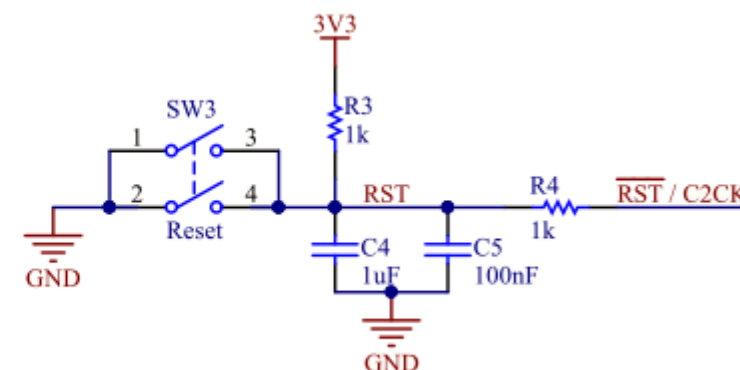
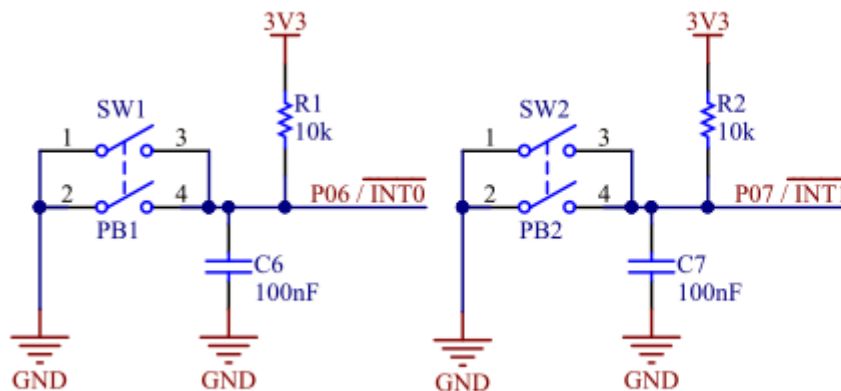
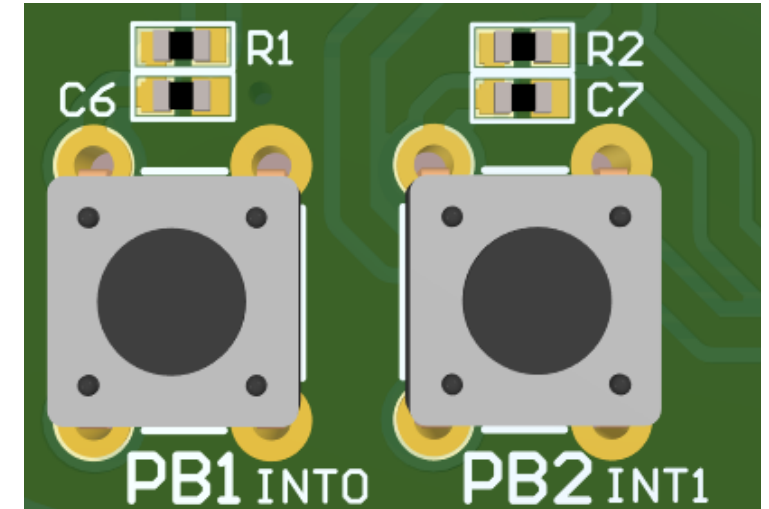
Lab. Sessions – Board Overview

- C8051F388
 - GPIO - All Pins can be accessed from the outside world!
 - Ports (P0-P4), UART, SPI, I2C...
 - Always check the board schematics and the user manual



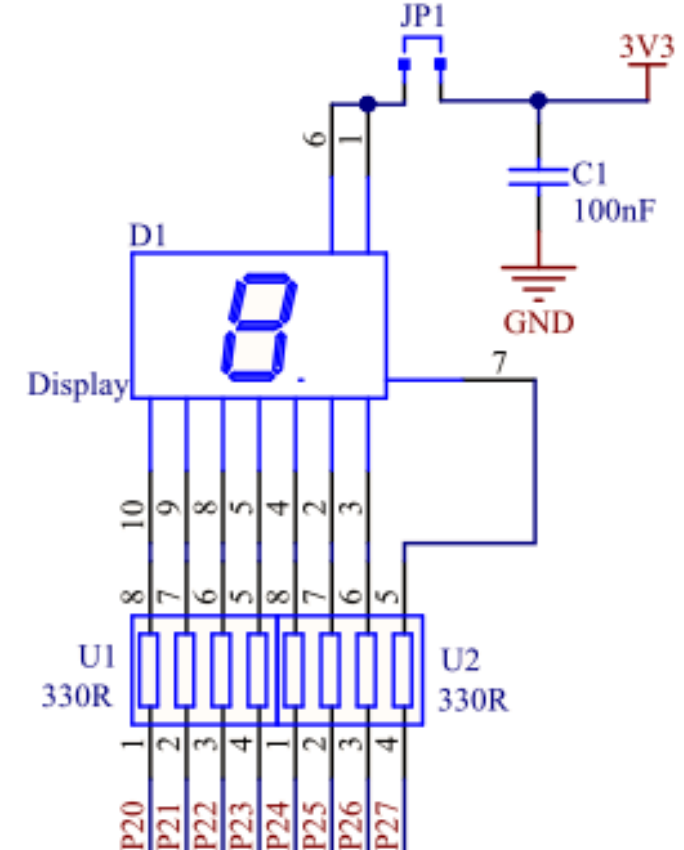
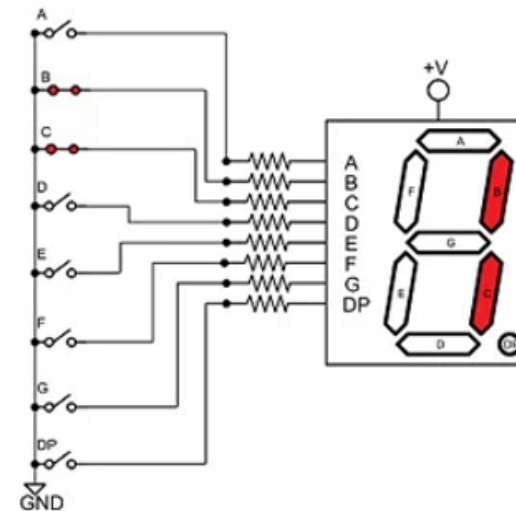
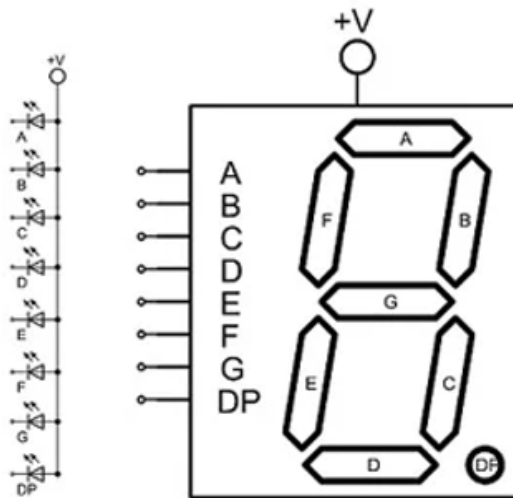
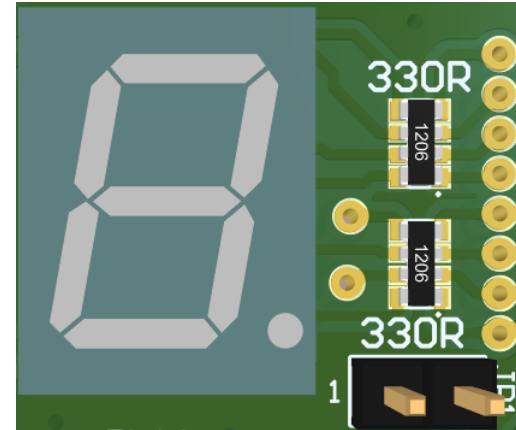
Lab. Sessions – Board Overview

- Push Buttons:
 - Check the logic:
 - NOT PRESSED, input is '1' – 3.3V
 - PRESSED, input is '0' – 0V (input is directly short-circuited with GND)
 - Think about the input logic for solving the assignment...



Lab. Sessions – Board Overview

- 7-seg display
 - Is just a set of leds;
 - Mapped on GPIO Port2;
 - Enabled with reverse logic;
 - Common anode display;



GPIO Configurations: Push-Pull vs Open-Drain

- GPIO Configuration:
 - While a push-pull sensor has two MOSFETs that alternately conduct to provide high or low output signals, an open drain sensor has only one MOSFET!
 - Push-pull is the most common output configuration.
 - PULL (to ground)...
 - PUSH (the power supply)...

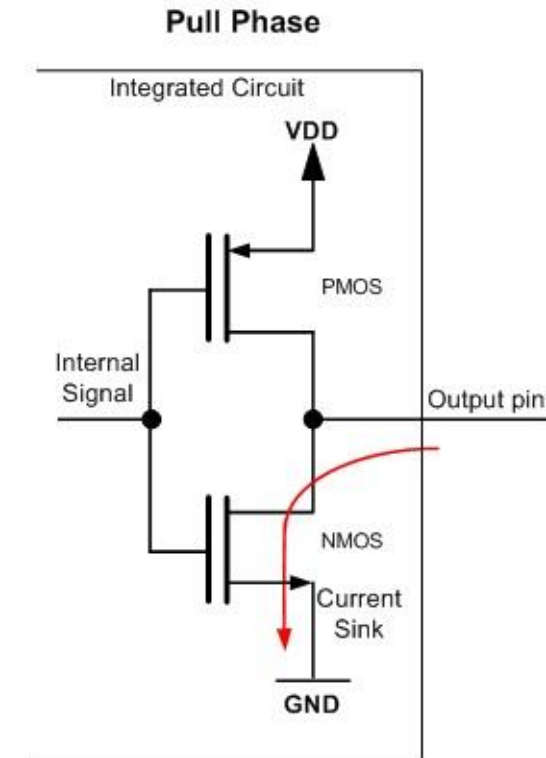
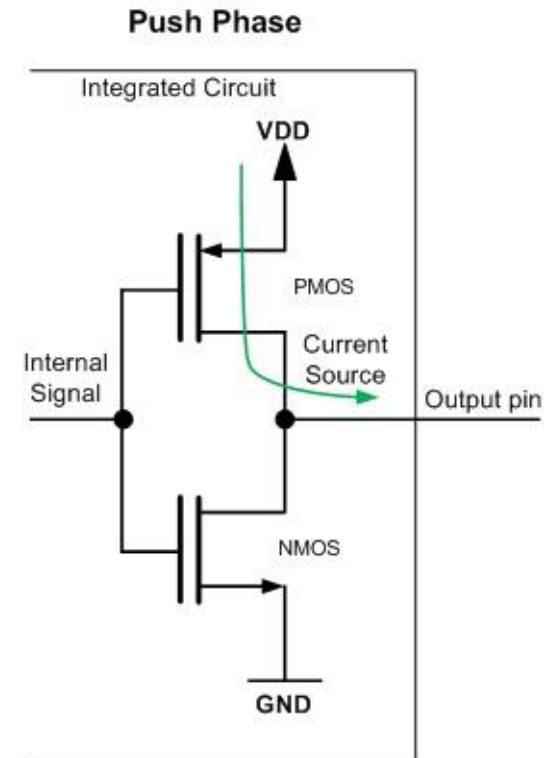
GPIO Configurations: Push-Pull vs Open-Drive

PUSH phase:

When the Internal Signal connected to the gates of the transistors is set to a low logic level (logic 0), the PMOS transistor is activated and current flows through it from the VDD to the output pin. NMOS transistor is inactive (open) and not conducting.

PULL phase:

When the Internal Signal connected to the gates of the transistors is set to a high logic level (logic 1), the NMOS transistor is activated (closed) and current starts to flow through it from the output pin to the GND. At the same time, the PMOS transistor is inactive (open) and is not conducting current.



GPIO Configurations: Push-Pull vs Open-Drain

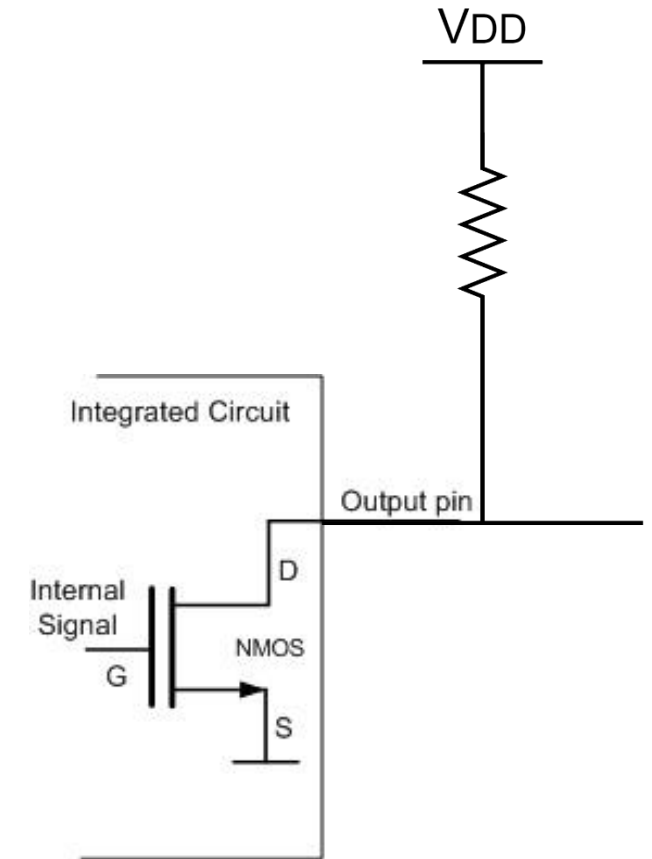
Open Drain:

In open drain configuration, the logic behind the pin can drive it only to ground (logic 0).

An N-channel MOS transistor pulls the output pin to ground when the transistor is on and leaves it floating when the transistor is off.

Mostly used in communication buses, e.g., I2C, one-wire...

When all of the outputs of the devices connected to the line are in Hi-Z state, the line is driven to a default logic 1 level by a pull-up resistor!



GPIO Configurations: Push-Pull vs Open-Drain

GPIO COnfiguration

To configure the GPIO, check the manual and use the config wizard tool...

Port I/O

Priority Crossbar | Ports 4 Input/Output

		P0								P1								P2								P3							
	Pin I/O	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
UART0	TX0																																
	RX0																																
	SCK																																
SPI0	MISO																																
	MOSI																																
	NSS																																
SMBus0	SDA																																
	SCL																																
CP0	CP0																																
	CP0A																																
CP1	CP1																																
	CP1A																																
SYSCLK	SYSCLK																																
PCA	CEX0																																
	CEX1																																
	CEX2																																
	CEX3																																
	CEX4																																
ECI	ECI																																
Timer0	T0																																
Timer1	T1																																
UART1	TX1																																
	RX1																																
SMBus1	SDA1																																
	SCL1																																
Analog / Digital -->		D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D		
Push-Pull / Open Drain -->		O	O	O	O	O	O	O	O	O	O	O	O	O	O	O	O	P	P	P	P	P	P	P	P	P	P	P	P	P	P		
Pin Skip -->																																	
P2MDOUT = 0xFF;																																	

Enable Crossbar
Disable Weak Pull-Up

P0.6 - X1
P0.7 - X2

P1.3 - ALE
P1.4 - CNVSTR
P1.5 - VREF
P1.6 - /RD
P1.7 - /WR

Assigned
Skipped

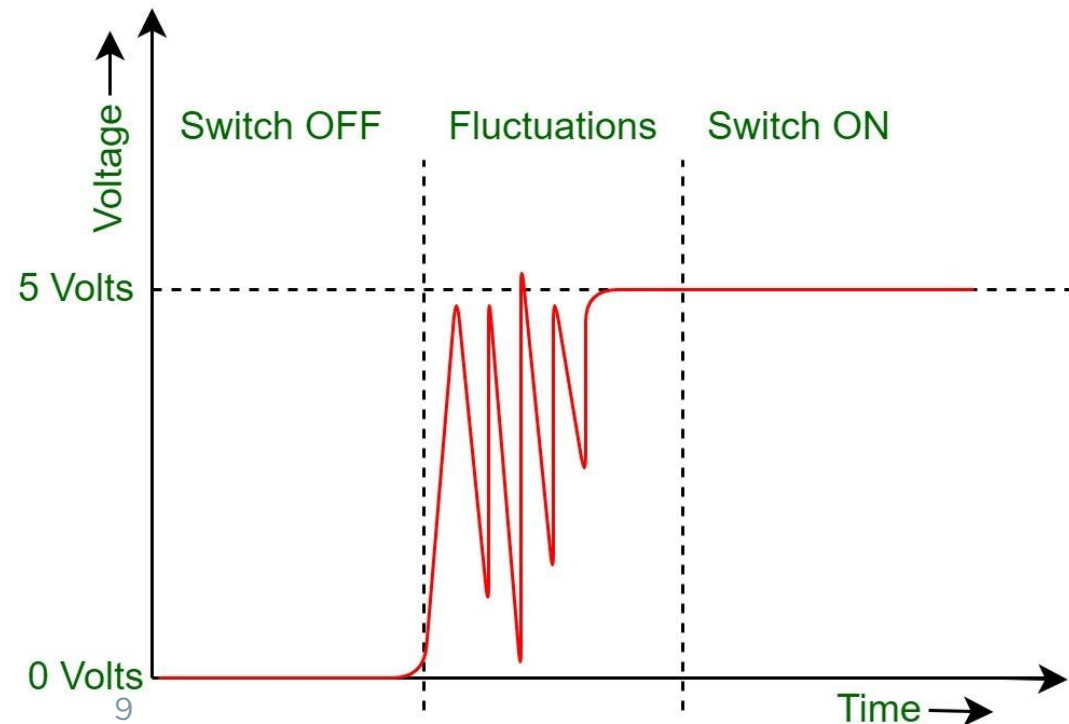
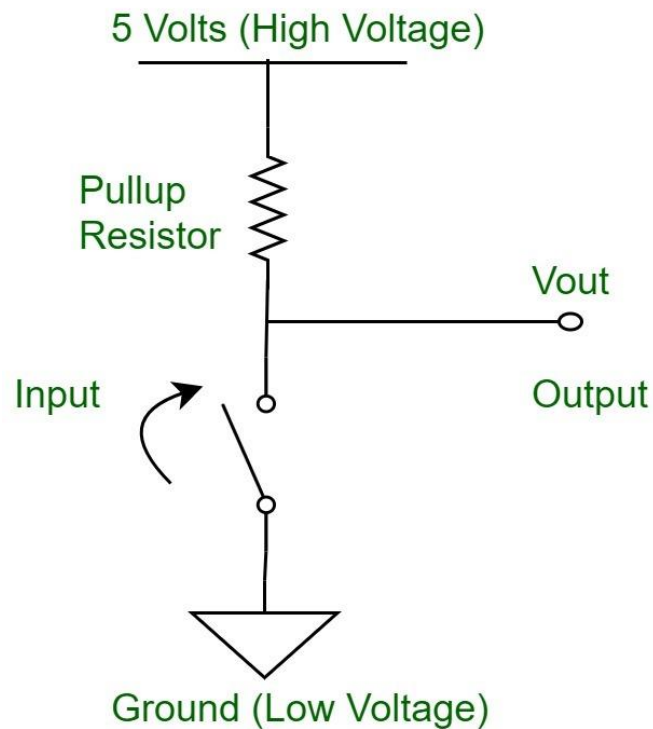
D - Digital
A - Analog

O - Open Drain
P - Push-Pull

OK Cancel Reset

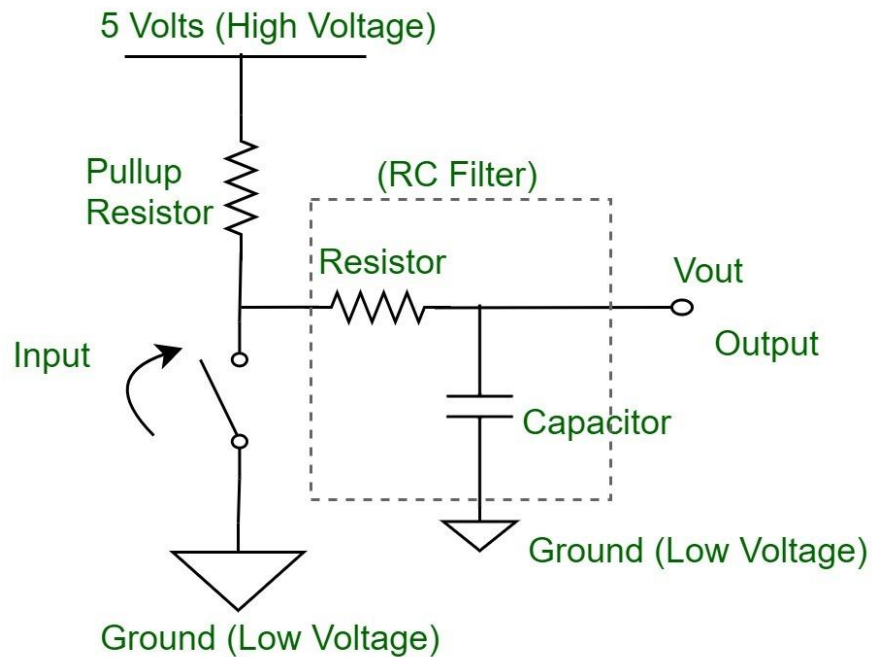
Bounce vs Debounce

- When using a GPIO pin with a push button...
 - Buttons have mechanical parts (switches/plates that connect to each other)
 - Mechanical parts are not perfect... and suffer from degradation over the time.
 - The mechanical design and materials used can cause spurious open/close transitions in the output signal when pressed, which may be read by the MCU as multiple presses...

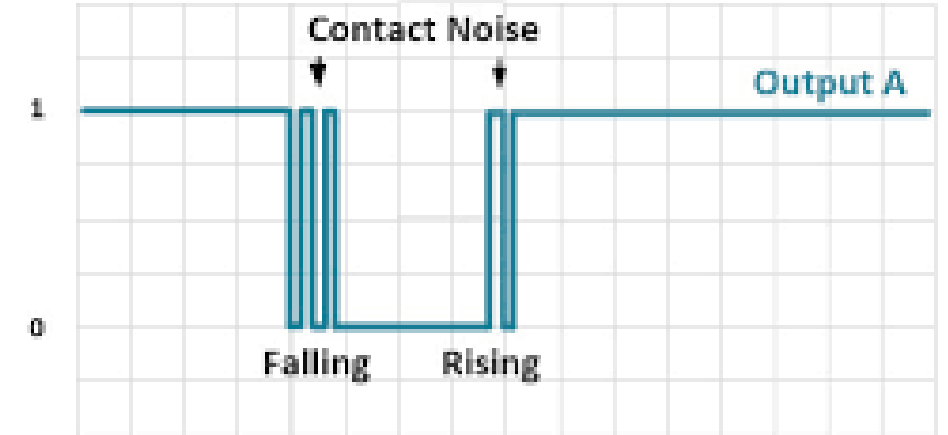


Bounce vs Debounce

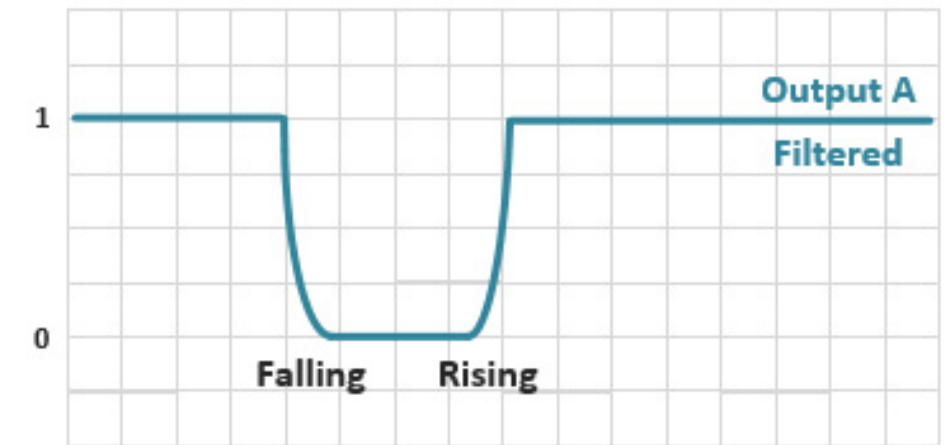
- When using a GPIO pin with a push button...
 - Hardware-based techniques include adding an RC filter...



Switch Debounce using RC Filter...



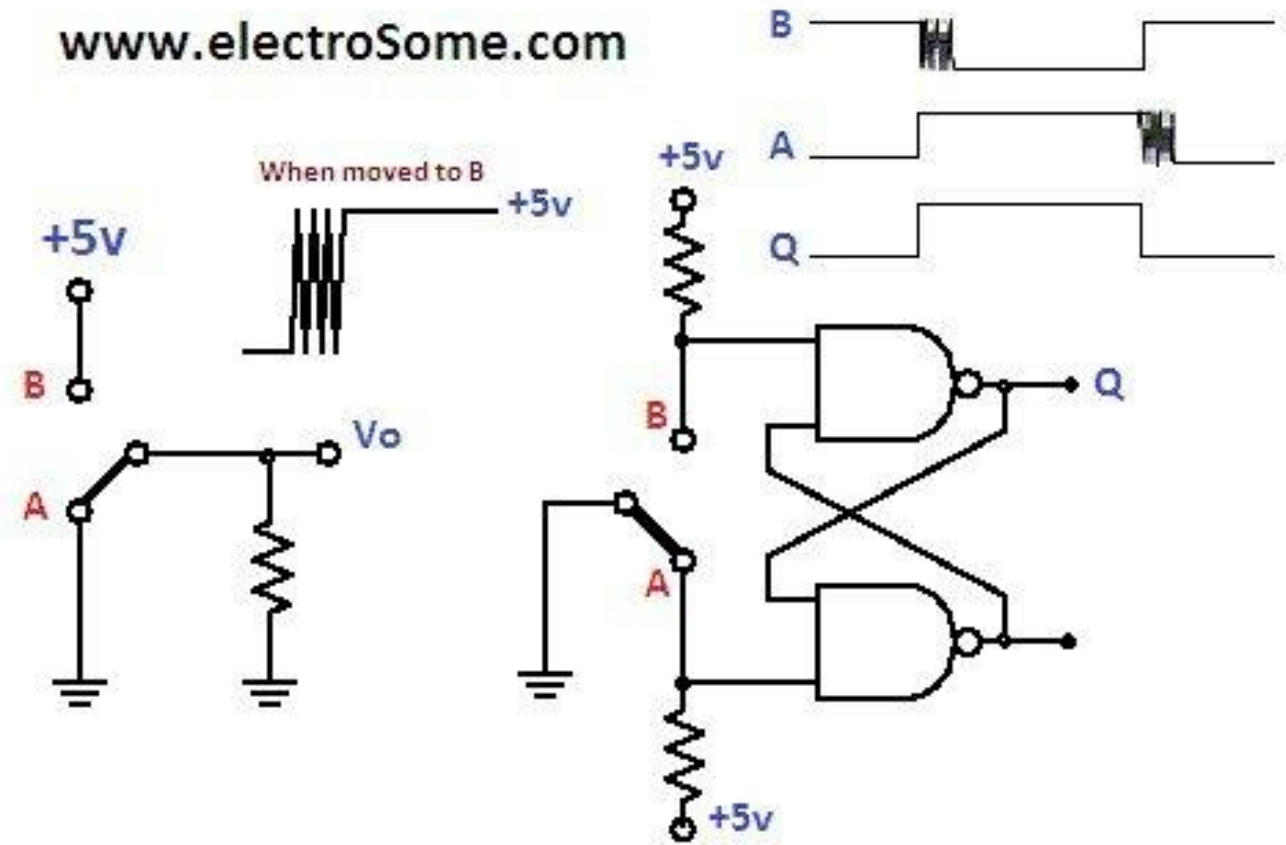
NO FILTER



WITH RC FILTER

Bounce vs Debounce

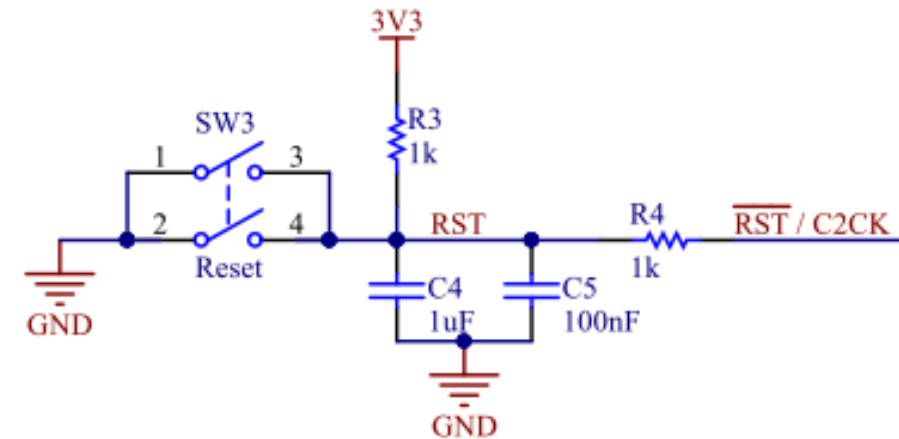
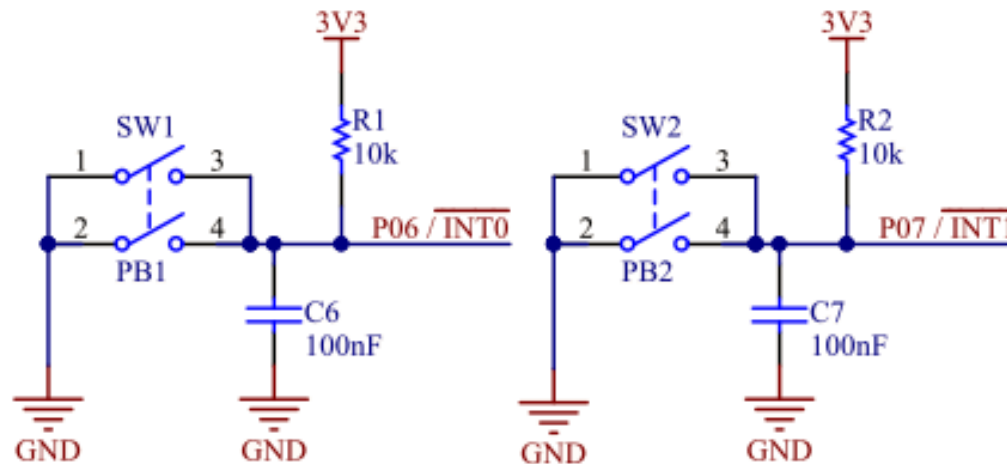
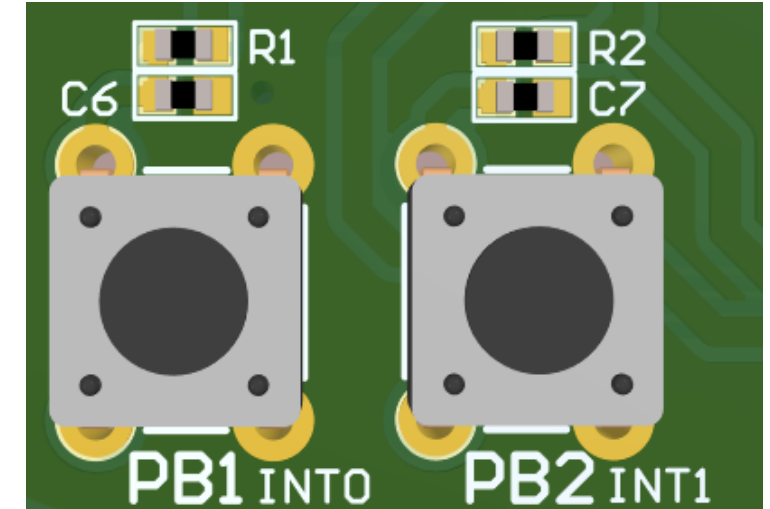
- When using a GPIO pin with a push button...
 - Using a SR flip-flop



Switch Debounce using SR Flip-Flop...

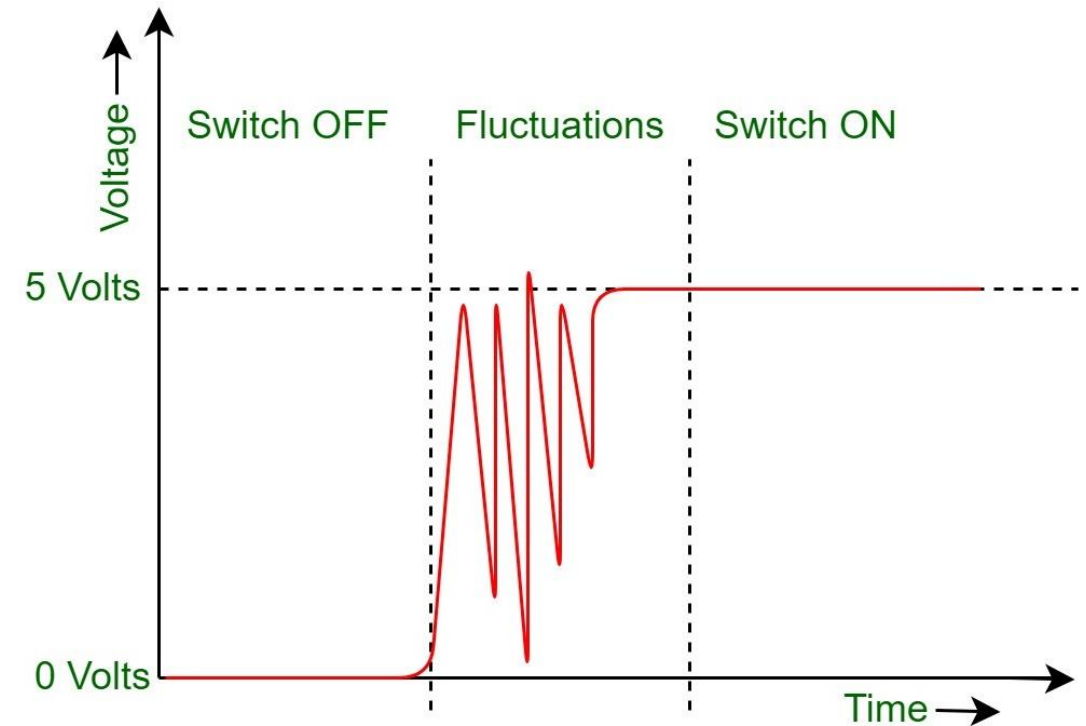
Bounce vs Debounce

- When using a GPIO pin with a push button...
 - Our board implements a debounce circuit in hardware
 - RC filter!



Bounce vs Debounce

- When using a GPIO pin with a push button...
 - However, the debounce circuit can be replaced by a software-only solution
 - It saves components (1 capacitor)
 - It can be more efficient than the hardware circuit



Bounce vs Debounce

- Software-based techniques:
 - How to do it?
 - Most mechanical switches today present a response/vibration time of $< 10\text{ms}$;
 - Human perception cannot perceive responses $< 100\text{ ms}$ (200 ms it is still tolerable and 50 ms is considered a really fast response);
- We can assume a debounce time of 100 ms adequate...

Bounce vs Debounce

- **Delay-based (Simple Delay):**

- After detecting a key press, the system waits for a predefined delay (e.g., 10-50 ms) before checking the state again. If the state is stable after the delay, it is considered a valid key press. This is simple but may introduce latency – blocking delay. Key state is only checked once.

- **State Change/Time-Based Debounce:**

- Tracks the state of the button and adds a timer to measure the time interval. When the button state changes, it starts the timer (debounce interval). If the state remains stable for the debounce period, it's considered a valid state change. This avoids unnecessary blocking delays but requires more logic (hardware timers). Key is only checked once.

Bounce vs Debounce

- **Counter-Based Debounce (integrator):**

- A counter is used to check how many consecutive stable states are observed. For example, if the system reads the same state for 10 consecutive cycles of 10 ms, it registers that state. You can use a soft delay or hardware timer to wait between the reads up to a debounce interval of < 100 ms.

- **Software Filtering with Shift Registers (sliding window):**

- Uses a shift register (8-bit variable) to keep track of recent button states. For example, an 8-bit shift register stores the button state from the last 8 samples. If all bits in the register match (e.g., all high or all low), the state is considered stable. You can use a software delay or hardware timer to wait between the reads up to a debounce interval of < 100 ms.

THANK YOU!

Questions?

mr.gomes@dei.uminho.pt